

Manuale Utente – QTMiner

1. Informazioni Generali

QTMiner è un'applicazione distribuita basata su architettura **client/server**, progettata per l'esecuzione dell'algoritmo di **clustering Quality Threshold (QT)** su dati presenti in un database **MySQL**.

Il sistema si compone di due componenti principali:

- **QTServer**: server multithread che gestisce la connessione al database, l'esecuzione dell'algoritmo di clustering e la comunicazione con i client.
- **QTClient**: applicazione testuale che consente all'utente di interagire con il server tramite un'interfaccia a menu.

L'obiettivo del progetto è fornire un sistema completo per l'analisi automatica di insiemi di dati attraverso l'algoritmo QT, mantenendo una struttura modulare e facilmente estendibile.

2. L'Algoritmo Quality Threshold (QT)

L'algoritmo **Quality Threshold (QT)** è un metodo di clustering che non richiede di specificare a priori il numero di cluster da ottenere.

Il criterio principale è la **distanza massima** (raggio R) tra i punti appartenenti a uno stesso cluster.

Funzionamento dell'algoritmo

1. L'utente definisce un raggio massimo R .
2. Per ogni tupla del dataset viene costruito un cluster candidato contenente tutte le tuple la cui distanza dal centroide è minore o uguale a R .
3. Tra tutti i cluster candidati viene scelto quello con il maggior numero di elementi, che diventa un **cluster reale**.
4. Le tuple assegnate a tale cluster vengono rimosse dal dataset.
5. Il processo viene ripetuto finché non rimangono tuple non assegnate.
6. Se tutte le tuple rientrano in un unico cluster, il sistema solleva un'eccezione (`ClusteringRadiusException`), segnalando che il raggio scelto è eccessivo.

3. Requisiti di Sistema

- Java | 17+ (build target 25)
- Maven | 3.8+
- MySQL | 8.x

4. Guida all'Installazione

1. Eseguire lo script SQL presente nella cartella `scripts/database/createdb.bat` / `scripts/database/createdb.sh`
2. Compilare con `buildsln.bat` / `buildsln.sh`
3. Avviare il server eseguendo da terminale `java -jar ./qtserver/target/qtserver-1.0.jar`
4. Avviare il client eseguendo da terminale `java -jar ./qtclient/target/qtclient-1.0.jar localhost 8080`

5. Funzionalità del Sistema

Il client comunica con il server mediante socket TCP, scambiando comandi codificati numericamente.

Le principali funzionalità sono:

- (0) Carica tabella dal database

Permette di selezionare una tabella esistente nel database MySQL e di caricarla come dataset per l'elaborazione.

- (1) Computa cluster dal database

Esegue l'algoritmo **Quality Threshold (QT)** sul dataset attualmente caricato in memoria, utilizzando un raggio scelto dall'utente.

- (2) Salva cluster su file

Consente di salvare il risultato del clustering calcolato in precedenza su un file `.dmp`

- (3) Computa cluster da file

Permette di ricaricare un clustering precedentemente salvato su file, evitando di rieseguire l'algoritmo su database.

6. Guida Utente

Connessione iniziale

All'avvio, il client stabilisce una connessione con il server, utilizzando l'indirizzo IP e la porta configurati.

Una volta connesso, viene mostrato un **menu testuale interattivo** con le varie opzioni disponibili.

Carica tabella dal database

1. Selezionare l'opzione **(0) Carica tabella dal database**.
2. Inserire il nome di una tabella presente nel database MySQL.
 - Se la tabella non esiste o non è corretta, viene visualizzato un messaggio di errore.
 - In caso positivo, il server conferma il caricamento e restituisce una descrizione del dataset.

Computa cluster dal database

3. Dopo il caricamento della tabella, selezionare **(1) Computa cluster dal database**.

4. Inserire il valore del raggio R (numero reale positivo).
 - Se R è non valido o minore/uguale a 0, verrà richiesto di reinserire il valore.
 - Se R è eccessivo, viene segnalato che tutti i dati rientrano in un unico cluster.
5. Al termine dell'esecuzione, il server restituisce:
 - Numero di cluster generati,
 - Centroidi di ciascun cluster,
 - Distanza media dei punti dal rispettivo centroide.

Esempio di output:

```
Number of Clusters: 3
0: Centroid=(sunny 30.3 high weak no)
Examples:
[sunny 30.3 high weak no] dist=0.0
[sunny 30.3 high strong no] dist=1.0
[sunny 13.0 high weak no] dist=0.570957095709571
AvgDistance=0.5236523652365236
```

Salva cluster su file

1. Selezionare l'opzione **(2) Salva cluster su file**,
2. Verrà creato un file `.dmp` in formato *nometabella_raggio*,
3. Il file verrà salvato sul server e potrà essere successivamente ricaricato.

Computa cluster da file

1. Selezionare l'opzione **(3) Computa cluster da file**,
2. Inserire il nome della tabella di riferimento,
3. Inserire il valore del raggio utilizzato per il calcolo del file da caricare,
4. Il server verifica l'esistenza del file e restituisce:
 - Numero di cluster generati,
 - Centroidi di ciascun cluster,
 - Distanza media dei punti dal rispettivo centroide.

In caso di errore (file inesistente o corrotto), viene visualizzato un messaggio appropriato.

7. Architettura del Progetto

Componenti principali del server

- `MultiServer` : gestisce le connessioni multiple dei client.
- `ServerOneClient` : gestisce la comunicazione con un singolo client.
- `QTMiner` : implementa l'algoritmo QT e coordina l'elaborazione dei dati.
- `Cluster` e `ClusterSet` : strutture dati per la rappresentazione dei cluster.
- `Data` , `Tuple` , `Attribute` : modellano il dataset e il calcolo delle distanze.

- `DBAccess`, `TableSchema`, `TableData`: gestiscono la connessione e l'interrogazione del database.

Componenti principali del client

- `MainTest`: interfaccia utente a menu testuale.
- `Keyboard`: utility per la lettura sicura dei dati da tastiera.

8. Esempio di Esecuzione

Esempio di sessione tipica tra client e server:

```
Server in ascolto sulla porta: 8080
Nuova connessione da: /127.0.0.1
[SERVER] Thread creato per /127.0.0.1

Avvio del client...
---MENU ---
(0) Carica tabella dal database
(1) Computa cluster dal database
(2) Salva cluster su file
(3) Computa cluster da file
Scelta (0/1/2/3):

Scelta (0/1/2/3): 0
Nome tabella: playtennis
outlook [overcast, rain, sunny], temperature, humidity [high, normal], wind [strong, weak], play [no, yes]
0: sunny, 30.3, high, weak, no
1: sunny, 30.3, high, strong, no
2: overcast, 30.0, high, weak, yes
3: rain, 13.0, high, weak, yes
4: rain, 0.0, normal, weak, yes
5: rain, 0.0, normal, strong, no
6: overcast, 0.1, normal, strong, yes
7: sunny, 13.0, high, weak, no
8: sunny, 0.1, normal, weak, yes
9: rain, 12.0, normal, weak, yes
10: sunny, 12.5, normal, strong, yes
11: overcast, 12.5, high, strong, yes
12: overcast, 29.21, normal, weak, yes
13: rain, 12.5, high, strong, no

Vuoi eseguire un'altra operazione? (s/n): s

---MENU ---
(0) Carica tabella dal database
(1) Computa cluster dal database
(2) Salva cluster su file
(3) Computa cluster da file
Scelta (0/1/2/3): 1
```

```

Cluster dal database:
Radius: 2
Number of Clusters: 3
0: Centroid=(sunny 30.3 high weak no)
Examples:
[sunny 30.3 high weak no] dist=0.0
[sunny 30.3 high strong no] dist=1.0
[sunny 13.0 high weak no] dist=0.570957095709571
AvgDistance=0.5236523652365236

1: Centroid=(overcast 12.5 high strong yes)
Examples:
[overcast 30.0 high weak yes] dist=1.5775577557755776
[overcast 0.1 normal strong yes] dist=1.4092409240924093
[sunny 12.5 normal strong yes] dist=2.0
[overcast 12.5 high strong yes] dist=0.0
[rain 12.5 high strong no] dist=2.0
AvgDistance=1.3973597359735974

2: Centroid=(rain 0.0 normal weak yes)
Examples:
[rain 13.0 high weak yes] dist=1.4290429042904291
[rain 0.0 normal weak yes] dist=0.0
[rain 0.0 normal strong no] dist=2.0
[sunny 0.1 normal weak yes] dist=1.0033003300330032
[rain 12.0 normal weak yes] dist=0.396039603960396
[overcast 29.21 normal weak yes] dist=1.964026402640264
AvgDistance=1.132068206820682

---MENU ---
(0) Carica tabella dal database
(1) Computa cluster dal database
(2) Salva cluster su file
(3) Computa cluster da file
Scelta (0/1/2/3): 2
Cluster salvati in file con successo

Vuoi eseguire un'altra operazione? (s/n): n

```

Autori

Sviluppo realizzato in collaborazione paritaria:

- **Mirco Catalano**
- **Lorenzo Amato**